

Architecture de l'Assistant d'Acquisition Gomecanicien (AAG)

Ce document décrit le fonctionnement technique du POC développé pour Gomécano.

1. Vision d'ensemble

L'objectif est de transformer des candidatures non structurées (PDF) en fiches de profils exploitables pour le recrutement de mécaniciens mobiles, et de les comparer automatiquement à un besoin opérationnel via une interface web interactive.

2. Le Pipeline de Données

Le flux de données suit trois étapes clés :

Ingestion (src/aag/ingestion/) :

- Lecture des fichiers PDF bruts (15 CV de démonstration)
- Conversion en texte (String Python)

Analyse et Structuration (src/aag/scoring/) :

- Recherche de mots-clés (Compétences, Ville, Expérience)
- Mapping vers un modèle de données JSON standardisé

Interface et Aide à la Décision (src/app_streamlit.py) :

- Lecture des profils JSON depuis data/samples_json/
- Configuration dynamique du besoin mission via la sidebar
- Calcul du score de compatibilité de 0 à 100 en temps réel
- Classement interactif des 15 candidats avec justifications
- Visualisation graphique des scores (Plotly)

3. Composants Techniques

- Langage : Python 3.1x
- Parsing PDF : PyPDF2 (léger, sans dépendances lourdes)
- Interface : Streamlit (dashboard interactif pour la démonstration)
- Visualisation : Plotly (graphique de scores en barres horizontales)
- Stockage : Fichiers JSON (pas de base de données complexe pour le POC)

4. Modèle de Données

Chaque Gomécanicien est représenté par cet objet JSON :

- nom : Identité du candidat
- ville : Zone d'intervention
- competences : Liste de compétences techniques (moteur, freinage, électrique, etc.)
- experience_annees : Nombre d'années d'expérience
- fichier_source : Nom du PDF d'origine

5. Arborescence du Projet

```
aag-gomecano-cv-matching/  
|-- data/  
|   |-- samples_cvs/          # 15 CV PDF de test  
|   |-- samples_json/        # 15 profils JSON extraits (sortie ingestion)  
|   |-- besoin.json           # Définition d'un besoin opérationnel  
|-- docs/  
|   |-- architecture.pdf      # Ce document  
|   |-- conception.pdf        # Conceptualisation globale du projet  
|   |-- demo_pitch_comex.pdf  # Script de présentation  
|   |-- scoring_rules.pdf     # Règles de scoring détaillées  
|-- scripts/  
|   |-- run_ingestion.py      # Point d'entrée extraction PDF -> JSON  
|   |-- run_demo.py           # Matching en ligne de commande (hors demo)  
|   |-- generate_sample_cvs.py # Génération des 15 CV PDF de demo  
|   |-- md_to_pdf.py          # Conversion documentation MD -> PDF  
|-- src/  
|   |-- aag/  
|   |   |-- ingestion/        # Lecture PDF + nettoyage + extraction  
|   |   |-- models/           # Structures de données  
|   |   |-- scoring/          # Logique de matching et règles métier  
|   |   |-- utils/            # Helpers (IO, logs)  
|   |-- app_streamlit.py       # Interface web de démonstration  
|-- tests/                     # Tests unitaires
```

6. Flux de la Démonstration

```
15 CV PDF          run_ingestion.py          15 JSON  
(samples_cvs/) --> (extraction) --> (samples_json/)  
                                     |  
                                     v  
                                app_streamlit.py  
                                (lecture + scoring  
                                + affichage)
```

La démo se fait entièrement via Streamlit. Le script `run_ingestion.py`

n'est relancé que si les CV sources changent.

7. Check-list de Validation

- [x] Arborescence complète créée
- [x] 15 CV PDF générés dans data/samples_cvs/
- [x] 15 profils JSON extraits dans data/samples_json/
- [x] Script run_ingestion.py fonctionnel
- [x] Interface Streamlit fonctionnelle
- [x] Documentation complète dans docs/